Chapitre 9- Les arbres

I Introduction

I.1 Utilité

Les arbres sont des structures de données très utilisées :

- dans les systèmes d'exploitation pour représenter l'organisation des répertoires et des fichiers
- en IA (Intelligence Artificielle) pour effectuer les raisonnements et définir les différentes stratégies (application dans les programmes de jeux)
- en algorithmique pour effectuer les tris
- en compilation pour la représentation des programmes (sous formes d'arbres)
- …

Chapitre 9- Les arbres

Donald Knuth, un des pionniers de l'informatique (à qui l'on doit l'ouvrage de référence sur l'algorithmique « The Art of Computer Programing ») considère les arbres comme " la structure la plus fondamentale de l'informatique ".

I.2 Intérêt en informatique

- ❑ les informations sont souvent hiérarchisées et peuvent être représentées sous formes arborescentes
- ❑ les structures de données arborescentes permettent le stockage de données volumineuses avec un accès efficace.

Chapitre 9- Les arbres

II Définition

- On appelle **arbre** une structure formée d'une racine et d'un ensemble fini (éventuellement *vide*) d'arbres appelés aussi sous-arbres.
- Un sous-arbre n'ayant pas lui-même de sous-arbres est appelé feuille. Exemple : dans un système d'exploitation, les feuilles correspondent aux fichiers.

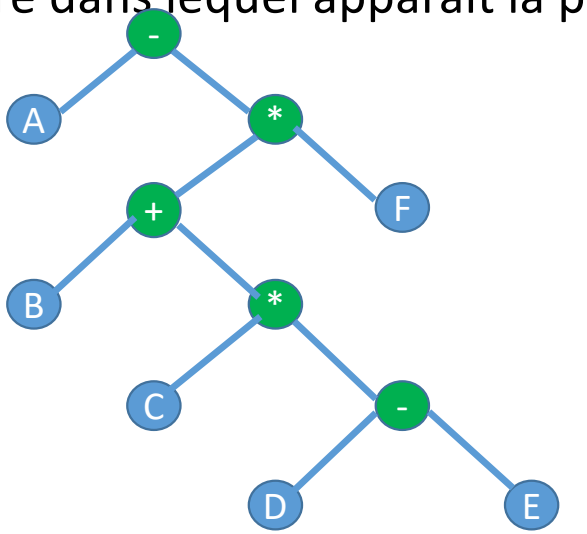
Un arbre peut être vu comme un *graphe* sans cycle avec des *nœuds* et des lignes (*arêtes*) les reliant.

Chapitre 9- Les arbres

Utilisation :

L'expression arithmétique : A - (B + C * (D - E)) * F

se décompose facilement en un arbre dans lequel apparaît la priorité des opérateurs :



Chapitre 9- Les arbres

II.1 Terminologie

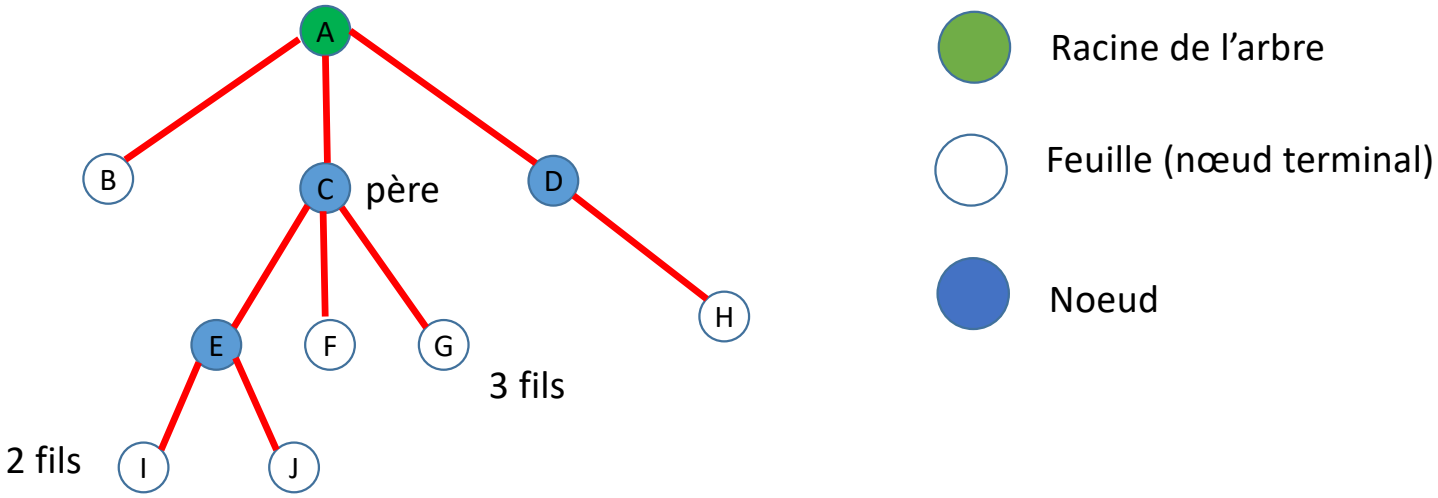
On distingue de ce fait trois types de **nœuds** :

- La **racine** de l'arbre : le seul **nœud** qui ne possède pas de *nœud père* (pas de prédécesseur)
- La **feuille** : c'est une *racine* sans *sous-arbre* (sans fils)
- **Nœud internes** : ce sont des *nœuds* qui ont des fils
- **Fils** : c'est un *sous-arbre* d'un arbre donné (arbre père)
- Les *sous-arbres* fils d'un même père sont dits arbres frères.
- *Feuilles* et *racine* sont des *nœuds* particuliers.

Représentation : on dessine l'arbre en partant de la racine (en haut) vers les feuilles (en bas).

Chapitre 9- Les arbres

Exemple :



Chapitre 9- Les arbres

Autres représentations :

❑ **Forme indentée :**

```
A
B
C
  E
    I
    J
  F
  G
D
  H
```

❑ **Forme parenthésée :**

(A (B, C (E (I, J), F, G) , D (H)))

Chapitre 9- Les arbres

II.2 Propriétés

Le ***degré*** d'un nœud est le nombre de ses fils.

La ***hauteur*** d'un nœud (ou arbre) est le nombre de nœuds sur la branche la plus longue, qui va de ce nœud aux feuilles qui en dépendent, plus 1. La hauteur d'un arbre correspond à celle de sa racine

La ***profondeur*** d'un nœud dans un arbre correspond au nombre de nœuds se trouvant sur le chemin qui va de la racine jusqu'à ce nœud.

*La racine d'un arbre a une profondeur de **1.***

*Les fils (ou sous-arbres) d'un nœud de profondeur **n** ont une profondeur de **n+1**.*

Le ***niveau*** d'un nœud : chaque nœud se trouve à un niveau de l'arbre. La racine est au niveau 0, ses fils au niveau 1, etc … Les nœuds d'un arbre qui ont la même profondeur sont au même niveau.

La ***taille*** d'un arbre est le nombre de nœuds qui le composent.

Un niveau est un ensemble de noeuds ou de feuilles situés à la même profondeur.

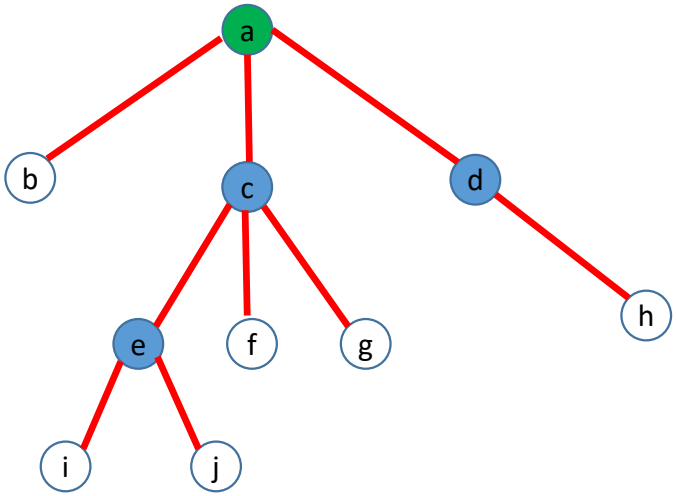
Un arbre vide a une hauteur de 0.

Un arbre avec une racine toute seule a une hauteur de 1.

Un arbre de degré **n** est dit arbre **n-aire.**

Chapitre 9- Les arbres

Exemple :



Profondeur(a) = 1
Profondeur(c) = 2
Profondeur(e) = 3
Hauteur(h) = 1
Hauteur(c) = 3
Hauteur(a) = 4
Degré(a) = 3
Degré(d) = 1
Degré(j) = 0
Taille(a) = 10

Chapitre 9- Les arbres

Remarques :

- Un arbre dont **tous les noeuds sont de degré 1** est appelé **arbre dégénéré** et c'est en fait une **liste**.

- Le degré d'un arbre est égal au plus grand des degrés de ses nœuds.

- Un arbre dont tous les nœuds sont nommés est dit **arbre étiqueté**. L'étiquette représente la "valeur" du noeud ou bien l'information qui lui est associée.

Chapitre 9- Les arbres

III Les arbres binaires

Ce sont des arbres d'au maximum degré 2 .

C'est un cas particulier d'arbres :

- les plus utilisés en informatique

  et

- les plus simples à étudier.

De plus, tout arbre non binaire peut être transformé (binarisé) en un arbre binaire.

Chapitre 9- Les arbres

III.1 Définition

Un arbre binaire est un arbre qui peut être défini récursivement comme suit :

- Soit vide
- Soit formé d'une racine
- Soit formé d'une racine et de un ou deux sous-arbres :
  - *sous-arbre gauche*

  et
  - *sous-arbre droit*

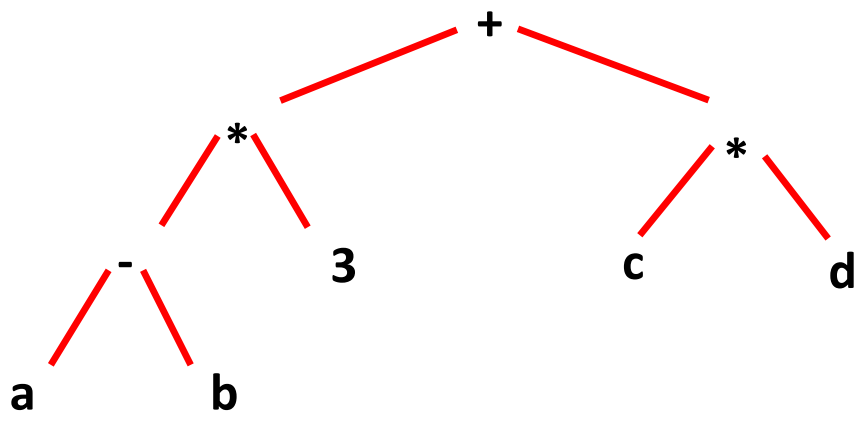
⇒ Le degré d'un nœud est égal à 0, 1 ou 2.

Chapitre 9- Les arbres

III.2 Exemple

Soit la représentation d'une expression arithmétique sous forme arborescente binaire :

$$((a - b) * 3) + (c * d)$$



Chapitre 9- Les arbres

- L'expression est représentée sans parenthèses
- Dans la décomposition apparaît la priorité des opérations
- Les feuilles sont des variables ou des valeurs constantes : a, b, 3, c, d
- La racine et les nœuds internes sont des opérateurs : -, *, +

Chapitre 9- Les arbres

III.3 Spécification

Soit **T_Arbre** le type arbre binaire contenant des éléments de type **T_Elt**.

Opérations :

- **fonction** Creation_Arbre **:** T_Arbre

-- *Création d'un arbre vide*

- **fonction** Arbre_Vide (A : T_Arbre) **:** Booléen

- **fonction** Val_Racine (A : T_Arbre) **:** T_Elt

- **fonction** Arbre_Droit (A : T_Arbre) **:** T_Arbre

- **fonction** Arbre_Gauche (A : T_Arbre) **:** T_Arbre

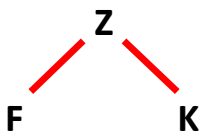
Chapitre 9- Les arbres

○ **fonction** Enraciner (La_Valeur : T_Elt; AG, AD : T_Arbre) **:** T_Arbre

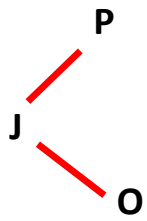
-- *AG : arbre gauche, AD : arbre droit*

Exemple : Soient les deux arbres A1 et A2 et la valeur G.

A1 :

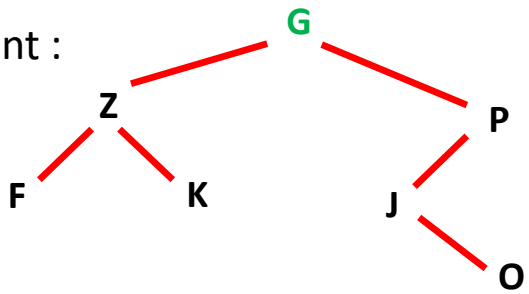


A2 :



A = Enraciner (**G**, A1, A2) donne l'arbre suivant :

A :



Chapitre 9- Les arbres

Remarque :

Si le langage de programmation dans lequel se fait l'implémentation le permet (exple : C++, Java, Ada) on peut surcharger l'opération

⇒ ***fonction*** *Enraciner(AG, AD : T_Arbre; La_Valeur : T_Elt)* ***:*** *T_Arbre*

○ **module** Place_Gauche(↕ A : T_Arbre; ↓ AG : T_Arbre)

-- A : arbre à greffer, AG : greffon

○ **module** Place_Droite(↕ A : T_Arbre; ↓ AD : T_Arbre)

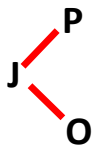
-- A : arbre à greffer, AD : greffon

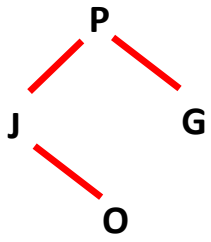
Chapitre 9- Les arbres

Exemple :

Place_Droite (A2, Enraciner (G, Creation_Arbre, Creation_Arbre)) donne l'arbre :

A2 :





○ **module** MAJ_Racine(↕ A : T_Arbre; ↓ Valeur : T_Elt)

○ **module** Detruire(↕ A : T_Arbre)

-- Détruit l'arbre en remontant des feuilles vers la racine

Chapitre 9- Les arbres

Quelques propriétés

Arbre_vide(Creation_Arbre) =

vrai

Val_Racine(Enraciner(Z, Creation_Arbre, Creation_Arbre)) =

Z

Val_Racine(Arbre_Gauche(Place_Gauche(Enraciner(F, Creation_Arbre, Creation_Arbre),
Enraciner(Z, Creation_Arbre, Creation_Arbre)))) =

Z

Chapitre 9- Les arbres

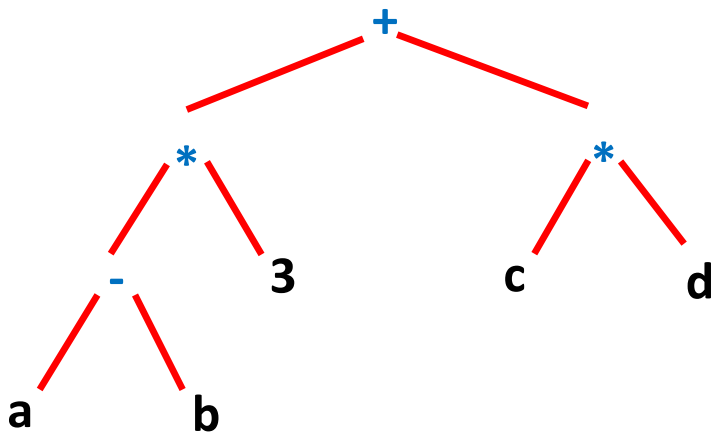
IV Intérêt des arbres binaires

IV.1 Analyse des expressions mathématiques

Exemple : Soit l'expression arithmétique :

$$((a - b) * 3) + (c * d)$$

Et sa représentation sous forme d'arbre :



On veut afficher cette expression.

Il existe 3 formes d'affichages possibles :

Chapitre 9- Les arbres

Forme 1 :

```
module Affiche (A : T_arbre)
si non Arbre_Vide(A) alors
|Ecrire(Val_Racine(A))
|Affiche(Arbre_Gauche(A))
|Affiche(Arbre_Droite(A))
fsi
```

Résultat :

+ * - a b 3 * c d

↔ notation **préfixée** ou notation **polonaise** ⇒ *opérateur opérande opérande*

Chapitre 9- Les arbres

Forme 2 :

module Affiche (A : T_arbre)

si **non** Arbre_Vide(A) **alors**

|Affiche(Arbre_Gauche(A))

|Affiche(Arbre_Droit(A))

|Ecrire(Val_Racine(A))

fsi

Résultat :

a b - 3 * c d * +

↔ notation ***post-fixée*** ou ***polonaise inversée*** => *opérande opérande opérateur*

Chapitre 9- Les arbres

Forme 3 :

module Affiche (A : T_arbre)

si **non** Arbre_Vide(A) **alors**

|Affiche(Arbre_Gauche(A))

|Ecrire(Val_Racine(A))

|Affiche(Arbre_Droit(A))

fsi

Résultat :

a - b * 3 + c * d

↔ notation ***infixée***=> *opérande opérateur opérande*

Chapitre 9- Les arbres

IV.2 Le tri

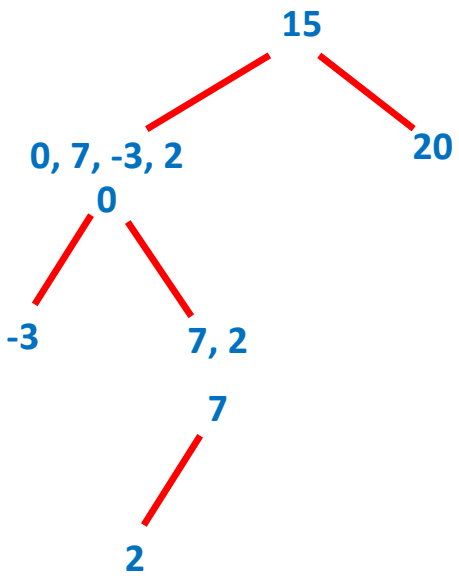
On peut utiliser un arbre binaire pour trier un ensemble de valeurs.

Principe :

- On place à gauche de la *racine* (on prend la première valeur) tout ce qui lui est **inférieur**
- On place à droite de la *racine* tout ce qui lui est **supérieur**
- On répète ce raisonnement sur chaque *sous-arbre*
- Pour obtenir l'ensemble **trié**, on effectue un **parcours infixé** de l'arbre

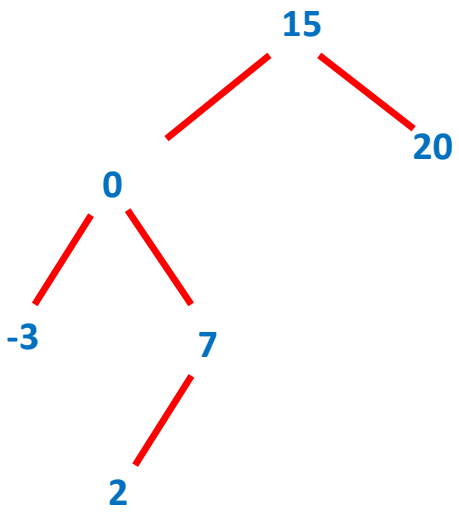
Chapitre 9- Les arbres

Exemple : 15, 0, 7, -3, 20, 2 : ensemble de valeurs à trier



Chapitre 9- Les arbres

Arbre résultat :



Après un *parcours infixé* de l'arbre on obtient l'ensemble trié : -3 0 2 7 15 20